

Deploying a Highly Available Apache Web Application on AWS

Architecture Overview

This guide demonstrates how to deploy a scalable and highly available Apache web application on AWS using:

- Custom VPC
- Public and Private Subnets
- Multi Availability Zone Architecture
- Application Load Balancer (ALB)
- Auto Scaling Group (ASG)
- Bastion Host
- NAT Gateway
- Ubuntu EC2 Instances
- Apache Web Server

The architecture hosts Apache servers inside private subnets while exposing the application securely through an internet-facing Application Load Balancer.

Final Architecture Flow

Users → Route 53 → Application Load Balancer → Apache EC2 Instances (Private Subnets) → Auto Scaling Group

Administrator → Bastion Host → SSH into Private Instances

Private Instances → NAT Gateway → Internet (for Apache installation and updates)

Complete Implementation Steps

Step 1 — Create the VPC

Create a custom VPC.

Configuration

Setting	Value
Name	Production-VPC

Setting	Value
IPv4 CIDR	10.0.0.0/16
Tenancy	Default

Step 2 — Create Public and Private Subnets

Create the following subnets:

Public Subnet A

Setting	Value
Name	Public-Subnet-A
CIDR	10.0.1.0/24
AZ	ap-south-1a

Public Subnet B

Setting	Value
Name	Public-Subnet-B
CIDR	10.0.2.0/24
AZ	ap-south-1b

Private Subnet A

Setting	Value
Name	Private-Subnet-A
CIDR	10.0.11.0/24
AZ	ap-south-1a

Private Subnet B

Setting	Value
Name	Private-Subnet-B
CIDR	10.0.12.0/24
AZ	ap-south-1b

Step 3 — Create and Attach Internet Gateway

Create an Internet Gateway and attach it to the VPC.

Configuration

Setting	Value
Name	Production-IGW

Step 4 — Create Public Route Table

Create a route table for public subnets.

Configuration

Setting	Value
Name	Public-RT
VPC	Production-VPC

Add Route

Destination	Target
0.0.0.0/0	Internet Gateway

Associate Subnets

Associate:

- Public-Subnet-A
- Public-Subnet-B

with:

- Public-RT
-

Step 5 — Create NAT Gateway

Create a NAT Gateway inside Public-Subnet-A.

Configuration

Setting	Value
Name	Production-NAT
Subnet	Public-Subnet-A
Elastic IP	Allocate New

Step 6 — Create Private Route Table

Create a route table for private subnets.

Configuration

Setting	Value
Name	Private-RT
VPC	Production-VPC

Add Route

Destination	Target
0.0.0.0/0	NAT Gateway

Associate Subnets

Associate:

- Private-Subnet-A
- Private-Subnet-B

with:

- Private-RT
-

Step 7 — Create Security Groups

ALB Security Group

Inbound Rules

Type	Port	Source
HTTP	80	0.0.0.0/0
HTTPS	443	0.0.0.0/0

Bastion Host Security Group

Inbound Rules

Type	Port	Source
SSH	22	Your Public IP

Web Server Security Group

Inbound Rules

Type	Port	Source
HTTP	80	ALB Security Group
SSH	22	Bastion Security Group

Step 8 — Launch Bastion Host

Launch an EC2 instance for Bastion Host.

Configuration

Setting	Value
AMI	Ubuntu Server 24.04 LTS
Instance Type	t2.micro
Subnet	Public-Subnet-A
Public IP	Enabled

Setting	Value
Security Group	Bastion-SG

Step 9 — Create Application Load Balancer

Create an internet-facing Application Load Balancer.

Configuration

Setting	Value
Scheme	Internet-facing
IP Type	IPv4
VPC	Production-VPC
Subnets	Public-Subnet-A + Public-Subnet-B
Security Group	ALB-SG

Step 10 — Create Target Group

Configuration

Setting	Value
Target Type	Instance
Protocol	HTTP
Port	80
VPC	Production-VPC
Health Check Path	/

Step 11 — Create Launch Template

Create a launch template for web servers.

Configuration

Setting	Value
AMI	Ubuntu Server 24.04 LTS
Instance Type	t2.micro
Security Group	WebServer-SG
Key Pair	Your Key Pair

User Data Script

```
#!/bin/bash
apt update -y
apt install apache2 -y
systemctl start apache2
systemctl enable apache2

echo "<h1>Apache Web Server from $(hostname)</h1>" > /var/www/html/index.html
```

Step 12 — Create Auto Scaling Group

Create an Auto Scaling Group using the Launch Template.

Configuration

Setting	Value
Launch Template	Apache-LT
VPC	Production-VPC
Subnets	Private-Subnet-A + Private-Subnet-B
Target Group	Apache-TG

Scaling Configuration

Setting	Value
Desired Capacity	2

Setting	Value
Minimum Capacity	2
Maximum Capacity	4

Step 13 — Test the Architecture

Open:

`http://ALB-DNS-Name`

You should see:

Apache Web Server from ip-xxxxx

Refresh multiple times to observe traffic distribution across instances.

Detailed Explanations of the Architecture

What is a VPC?

A VPC (Virtual Private Cloud) is an isolated private network inside AWS.

It allows you to define:

- IP ranges
- routing
- subnet architecture
- internet access
- security boundaries

The VPC acts like your own data center inside AWS.

Why Use Public and Private Subnets?

Public Subnets

Public subnets contain resources that require direct internet connectivity.

Examples:

- Application Load Balancer

- Bastion Host
- NAT Gateway

Public subnets have routes pointing to the Internet Gateway.

Private Subnets

Private subnets contain internal application servers.

These servers:

- do not receive public IPs
- are hidden from the internet
- can only be accessed internally

This improves security.

Why Is the ALB in Public Subnets?

Users from the internet must reach the Application Load Balancer.

Therefore:

- ALB needs internet access
- ALB must exist in public subnets
- ALB spans multiple AZs for high availability

AWS requires at least:

- 2 subnets
- in 2 different Availability Zones

when creating an Application Load Balancer.

Why Are Web Servers in Private Subnets?

Web servers should not be directly exposed to the internet.

Instead:

User Traffic → ALB → Private EC2 Instances

This creates a secure architecture.

Only the ALB communicates with the servers.

What is a Bastion Host?

A Bastion Host is a jump server.

Administrators first SSH into the Bastion Host.

Then from there:

- SSH into private instances
- manage application servers securely

Without a Bastion Host:

- private servers cannot be accessed directly
-

Why Is NAT Gateway Needed?

Private EC2 instances still require outbound internet access.

Examples:

- installing Apache
- downloading packages
- system updates
- pulling dependencies

However:

- they should NOT be publicly accessible

NAT Gateway solves this.

It allows:

Private Instance → Internet

but blocks:

Internet → Private Instance

What Does a Route Table Do?

Route tables decide where network traffic should go.

Example:

Destination	Target
0.0.0.0/0	Internet Gateway

means:

"Any traffic going outside the VPC should go to the internet gateway"

Why Do Private Subnets Use NAT Gateway?

The private route table contains:

Destination	Target
0.0.0.0/0	NAT Gateway

Meaning:

Private instances use NAT Gateway whenever they need internet access.

What is an Auto Scaling Group?

Auto Scaling Group automatically:

- launches new EC2 instances
- replaces failed instances
- scales infrastructure during traffic spikes

This improves:

- scalability
 - fault tolerance
 - automation
-

Why Use Multiple Availability Zones?

Availability Zones are separate AWS data centers.

If one Availability Zone fails:

- the application continues running in the other AZ
- users experience minimal downtime

This is critical for production applications.

How Traffic Flows Through This Architecture

User Request Flow

User → Route 53 → Application Load Balancer → Apache Web Servers

Administrative SSH Flow

Administrator → Bastion Host → Private EC2 Instance

Internet Access Flow for Private Servers

Private EC2 Instance → NAT Gateway → Internet

Final Outcome

After completing this setup, you will have:

- Highly available web architecture
- Load balanced application
- Secure private application servers
- Multi-AZ deployment
- Automatic scaling
- Production-style AWS infrastructure
- Apache web application running securely

This architecture closely resembles real-world enterprise deployments on AWS.